

前言

01.学习概述

1.1 学习目标

1.2 前置知识

02.核心概念

2.1 是什么？

2.2 解决什么问题？

2.3 基本特性

03.原理机制

3.1 基础知识_OpenGL ES坐标系

3.2 GPU渲染流程

3.3 OpenGL 着色器语言 GLSL

3.4 Android OpenGL渲染流程_其一：代码架构设计

3.5 Android OpenGL渲染流程_其二：TriangleDrawer实现

3.6 Android OpenGL渲染流程_其三：完整代码

3.3 Android OpenGL渲染流程_其四：纹理基础知识

3.3 Android OpenGL渲染流程_其五：使用纹理显示图片

3.5 绘制模式

04.底层原理

05.深度思考

5.1 关键问题探究

5.2 设计对比

06.实践验证

6.1 行为验证代码

6.2 性能测试

07.应用场景

7.1 最佳实践

7.2 使用禁忌

08.总结提炼

8.1 核心收获

8.2 知识图谱

8.3 延伸思考

09.参考资料

其他介绍

01.关于我的博客

前言

学习要符合如下的标准化链条：了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

01.学习概述

• 学习主题：

• 知识类型：

☐ ☒ Android/

☐ ☒ 01.基础组件与机制

☐ ☒ 四大组件

☐ ☒ IPC机制

- ☐ ☒ 消息机制
- ☐ ☒ 事件分发机制
- ☐ ☒ View与渲染体系 (含Window、复杂控件、动画)
- ☐ ☒ 存储与数据安全 (SharedPreferences/DataStore/Room/Scoped Storage)
- ☐ ☒ 02. 架构与工程化
 - ☐ ☒ 架构模式 (MVC/MVP/MVVM/MVI)
 - ☐ ☒ 依赖注入 (Koin/Hilt/Dagger)
 - ☐ ☒ 路由与模块化 (ARouter、Navigation)
 - ☐ ☒ Gradle与构建优化
 - ☐ ☒ 插件化与动态化
 - ☐ ☒ 插桩与监控框架
- ☐ ☒ 03.性能优化与故障诊断
 - ☐ ☒ ANR分析与优化
 - ☐ ☒ 启动耗时优化
 - ☐ ☒ 内存泄漏监控
 - ☐ ☒ 监控与诊断工具
- ☐ ☒ 04.Jetpack与生态框架
 - ☐ ☒ Room
 - ☐ ☒ Paging
 - ☐ ☒ WorkManager
 - ☐ ☒ Compose
- ☐ ☒ 05.Framework与系统机制
 - ☐ ☒ ActivityManagerService (含ANR触发机制)
 - ☐ ☒ Binder机制
- ☐ ☒ 音视频开发/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.OpenGL渲染视频
 - ☐ ☒ 03.FFmpeg音视频解码
- ☐ ☒ Java/
 - ☐ ☒ 01.基础知识
 - ☐ ☒ 02.集合框架
 - ☐ ☒ 03.异常处理
 - ☐ ☒ 04.多线程与并发
 - ☐ ☒ 06.JVM
- ☐ ☒ Kotlin/
 - ☐ ☒ 01.基础语法
 - ☐ ☒ 02.高阶扩展
 - ☐ ☒ 03.协程和流
- ☐ ☒ Flutter/

- ☐ ☒ 01.基础基础语法
- ☐ ☒ 02.状态管理
- ☐ ☒ 03.路由与依赖注入
- ☐ ☒ 04.原生通信
- ☐ ☒ 自我管理/
 - ☐ ☒ 01.内观
- ☐ ☒ 项目经验/
 - ☐ ☒ 01.启动逻辑
 - ☐ ☒ 02.云值守
 - ☐ ☒ 03.智控平台

- 学习来源:
- 重要程度: ★★☆☆☆
- 学习日期: 2025.
- 记录人: @panruiqi

1.1 学习目标

- 了解概念->探究原理->深入思考->总结提炼->底层实现->延伸应用"

1.2 前置知识

- ☐ OpenGL:

02.核心概念

2.1 是什么?

前言:

- 我们回顾之前的流程，整个视频的播放流程是什么样的呢？
- 解码，渲染。我们前面看了解码了，我们有了对应的YUV帧，那么什么是渲染呢？
- 渲染本质就是计算出每个像素该用什么样的色彩（RGB）。好，也就是把我们解码出的YUV帧解析为RGB帧。这个很容易，就是简单的加减乘除运算，这是他们的换算公式

RGB和YUV的换算

$$\begin{aligned} Y &= 0.299R + 0.587G + 0.114B \\ U &= -0.147R - 0.289G + 0.436B \\ V &= 0.615R - 0.515G - 0.100B \end{aligned}$$

。

$$\begin{aligned} R &= Y + 1.14V \\ G &= Y - 0.39U - 0.58V \\ B &= Y + 2.03U \end{aligned}$$

为什么要用OpenGL，OpenGL是什么？

- 我们上面的渲染当然可以用cpu，但是如果是1980 * 1080这种百万，千万级别的像素，用cpu处理起来就太慢了，就像几个大学生不断做乘除题目。幸好我们有gpu，许多个小学生，让他们做乘除题目。用GPU来进行渲染实现硬件加速。
- 我们跟cpu沟通容易，因为CPU可以直接执行机器语言（01指令），但GPU有自己专用的指令集，我们不能直接控制他们啊。OpenGL就是一个API，供我们访问控制GPU的API，我们可以通过他来控制GPU执行相关的渲染任务。
- 好，现在引出了OpenGL ES概念，其全称：OpenGL for Embedded Systems，是OpenGL 的子集，是针对手机 PAD等小型设备设计的，删减了不必须的方法、数据类型、功能，减少了体积，优化了效率。

我们知道OpenGL是我们控制GPU执行渲染操作的API，那么GPU的渲染过程是什么样的呢？

- 现代化的渲染流程如下
 - 顶点处理 (Vertex Processing)：核心是在屏幕上确定物体的几何形状。
 - 图元装配 (Primitive Assembly)：把顶点按渲染指令组合成图元（通常是三角形），比如一个矩形由两个三角形组成。
 - 光栅化 (Rasterization)：把几何形成切成一个个的细小的片元，每个片元对应屏幕上的一个像素候选。
 - 片元处理 (Fragment Processing / Pixel Shading)：在纹理坐标上取颜色。经过着色器计算最终片元颜色。
 - 测试与混合 (Tests & Blending)：判断该片元是否可见。根据需要丢弃部分片元。
 - 写入帧缓冲：通过上面的处理，最终可见的片元颜色写入屏幕（或 FBO）

2.2 解决什么问题？

如何和GPU通信，让其执行渲染的任务

2.3 基本特性

03.原理机制

3.1 基础知识_OpenGL ES坐标系

在音视频开发中，涉及到的坐标系主要有三个：Android屏幕坐标系，这是我们直观感受到的，手指触碰到的坐标系。然后是世界坐标和纹理坐标。

先来看看Android的屏幕坐标系

- 这是Android的屏幕坐标系，坐标原点在左上角
 - 原点：左上角 (0, 0)
X 轴：向右增长
Y 轴：向下增长 ← 注意
单位：像素 (px)
范围：[0, screenWidth] × [0, screenHeight]

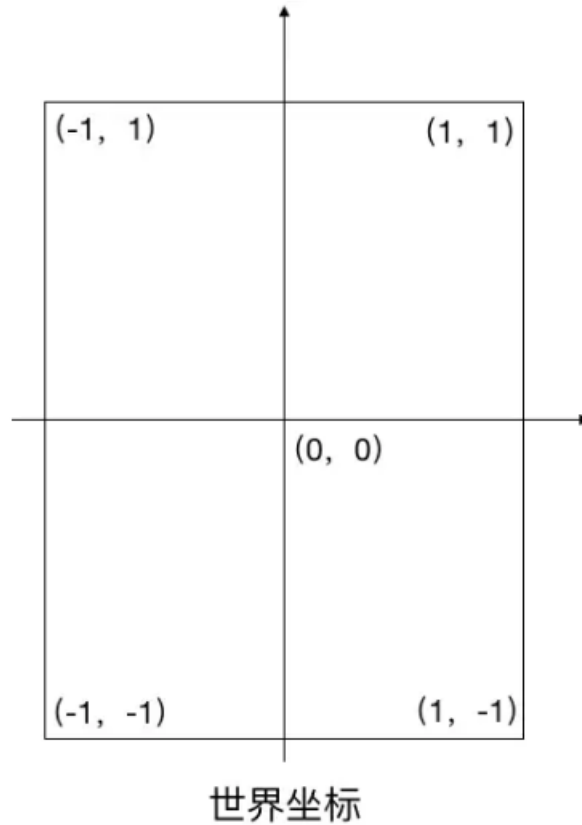
示例（1080×2340 屏幕）：

```
| (0,0)                (1080,0) |
|                                |
|          屏幕中心          |
|                                |
```

(540, 1170)
(0, 2340) (1080, 2340)

再看看世界坐标系，也就是Android屏幕中各点在顶点处理时，OpenGL视角中的位置

- 如下：坐标原点在屏幕中心，那么屏幕中的540,1170；这里就是0,0



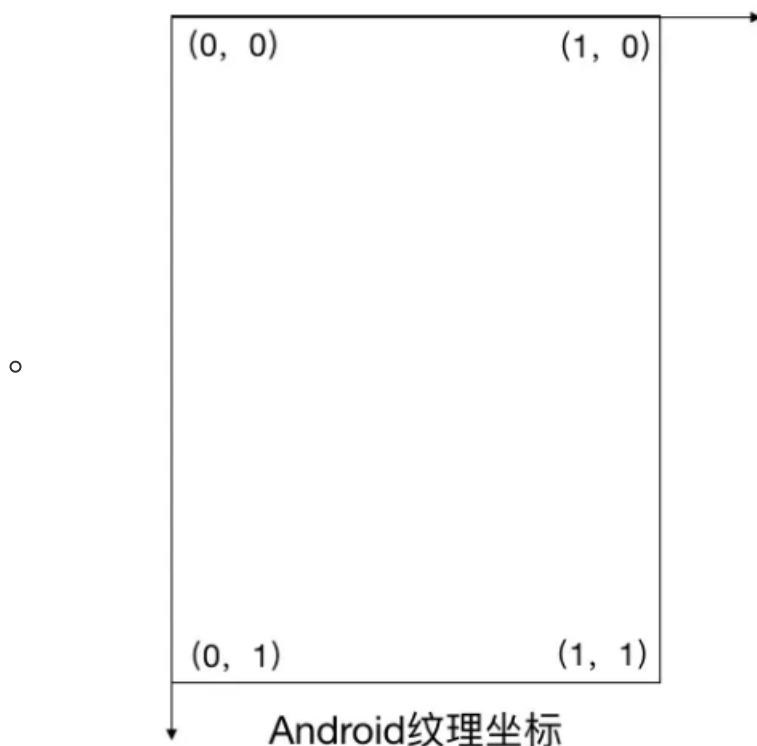
- 原点：中心 (0, 0)
X 轴：向右增长
Y 轴：向上增长 ← 注意：与 Android 相反！
单位：归一化（无单位）
范围：[-1, 1] × [-1, 1]

示例：

(-1, 1)	(1, 1)
原点 (0, 0)	
(-1, -1)	(1, -1)

再来看纹理坐标，纹理坐标就是Android屏幕中各点在片元处理时，OpenGL视角中的位置

- 纹理坐标：OpenGL ES 渲染时，有了要渲染的点后，还要知道要渲染的颜色，纹理坐标上的一个点代表在这个点渲染什么色彩。



3.2 GPU渲染流程

好，假设我们要绘制一个纯色的三角形，以其举例。

第一步：cpu侧首先得准备顶点坐标，纹理坐标和色彩数据三个数据（纯色的，不用贴图）

- 如下

```
// 假设要绘制一个三角形
val vertices = floatArrayOf(
    // 顶点位置 (x, y)    纹理坐标 (u, v)    颜色 (r, g, b)
    -0.5f, -0.5f,        0.0f, 0.0f,        1.0f, 0.0f, 0.0f, // 顶点0: 左下, 红色
    0.5f, -0.5f,         1.0f, 0.0f,        0.0f, 1.0f, 0.0f, // 顶点1: 右下, 绿色
    0.0f, 0.5f,          0.5f, 1.0f,        0.0f, 0.0f, 1.0f // 顶点2: 顶部, 蓝色
)
```

第二步：数据要被送到GPU的显存中

第三步：顶点着色器接收数据，并进行相关处理，给出空间变化后的输出

- 这里gl_Position = a_Position是指不经过空间变化。如果经过空间变化，让其乘以矩阵即可。

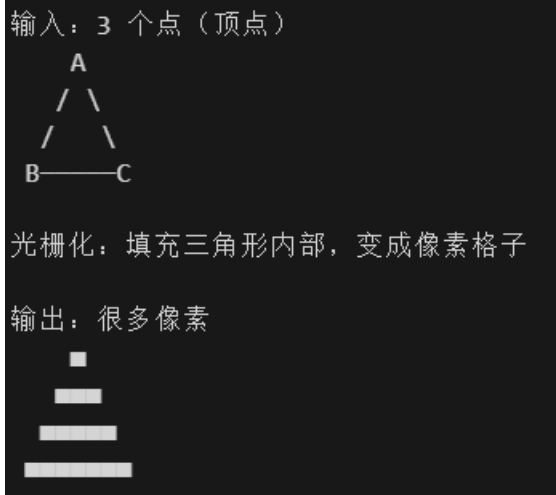
```
// 顶点着色器
attribute vec4 a_Position; // 输入：位置
attribute vec2 a_TexCoord; // 输入：纹理坐标

varying vec2 v_TexCoord; // 输出：传递纹理坐标

void main() {
    gl_Position = a_Position; // ✓ 输出1：位置
    v_TexCoord = a_TexCoord;  // ✓ 输出2：纹理坐标
}
```

第四步：GPU自身接收顶点着色器的输出数据，对其进行光栅化

- 光栅化负责把若干个顶点坐标切成成千上万个像素，并通过插值计算每个像素对应的纹理坐标（也就是其所拥有的颜色）类似这个



- 这是插值计算过程，也就是屏幕总哪些像素位于区域内，他们对应的色彩需要去纹理坐标哪个位置采样？

步骤1：判断哪些像素在三角形内
扫描屏幕，发现 1000 个像素在三角形里面

步骤2：对每个像素，算出它的纹理坐标

- 像素(0, 50) → 算出纹理坐标 (0.5, 0.5) ← 用插值算的
- 像素(10, 30) → 算出纹理坐标 (0.6, 0.3) ← 用插值算的
- 像素(-20, 40) → 算出纹理坐标 (0.3, 0.4) ← 用插值算的
- ... (共 1000 个)

1000 个像素，每个带着它的纹理坐标：

- 像素 0：屏幕位置(0, 50)，纹理坐标(0.5, 0.5)
- 像素 1：屏幕位置(10, 30)，纹理坐标(0.6, 0.3)
- 像素 2：屏幕位置(-20, 40)，纹理坐标(0.3, 0.4)
- ...

第五步：片元着色器渲染

- 上面把插值计算的纹理坐标送进来，对于每个像素，拥有其纹理坐标后，就可以从图片上取色，放置在对应的屏幕位置。这里我们有对应像素的屏幕位置，以及其需要的纹理坐标。我们只需要去纹理坐标中取色，然后放到屏幕的对应位置像素上即可
 - 最终，我们得到了一个存储各像素点对应的色彩值的数据。由此，我们可以控制硬件在屏幕像素的各个位置显示对应的色彩，但是这是需要底层硬件能力支持的，我们最后还要放到屏幕上

```
// 片元着色器
uniform sampler2D u_Texture; // 纹理图片
varying vec2 v_TexCoord;    // 接收：插值后的纹理坐标

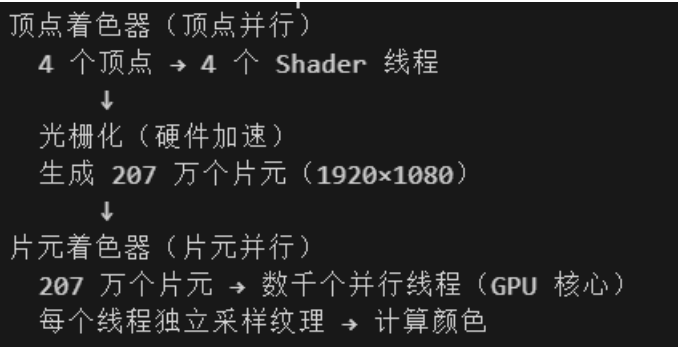
void main() {
    // 用纹理坐标去图片上取颜色
    gl_FragColor = texture2D(u_Texture, v_TexCoord); // ✅ 这里才取颜色！
}
```

- 怎么就放到屏幕上了？举例理解一下吧，比如我们有一个GLSurfaceView，他其实就是SurfaceView + EGL。
 - Surface创建时，由SurfaceFlinger分配给Surface一连串的GraphicBuffer。（这个是公有的）
 - EGL会创建一个默认的FBO，FBO关联到这一连串的GraphicBuffer中，同时通过eglMakeCurrent于让OpenGL知道EGLContext以及GPU的最终渲染目标
 - 通过eglSwapBuffers告诉OpenGL控制GPU渲染数据到GraphicBuffer中

- 渲染完成后，通过Binder告知SurfaceFlinger进程获取渲染结果，系统服务会自动将它混合显示到屏幕上

提问：我觉得直接一个坐标系，存放1080 * 2340的数据，然后坐标上存放色彩是最好的选择，色彩值为空，代表不渲染这个点，存放色彩值代表渲染，这样效率不是更高吗？一个坐标系，既可以决定在哪渲染，也可以决定渲染什么色彩。

- 把“颜色直接存在世界坐标”中，实际上是在描述**整个屏幕每个像素的颜色**，也就是说你直接生成了最终渲染结果，这样相当于你放弃了矢量式的描述（顶点+纹理）以及其高效处理，直接提前生成最终像素。这不仅浪费内存（尤其在 4K、8K 时代），也失去 GPU 逐像素计算和重用的能力。
- GPU其实只需要少量几何数据+纹理，就可以通过光栅化将少量顶点组成的几何形状切分为几十万个像素，同时通过插值计算出他们的纹理坐标，并自动取样渲染。这个是GPU提供的能力。执行过程如下：



- 这是他们的对比

方案	数据结构	内存占用	灵活性	GPU 利用率
分离方案 (现代 GPU 管线)	顶点数组 + 纹理图片	4 个顶点 + 1 张纹理 (例如: 32 字节 + 2MB)	★★★★★ 可变形、换纹理、缩放	★★★★★ 光栅化、插值、并行
合并方案 (直接存像素颜色)	每个像素的颜色	1920×1080×4 = 8.3MB (每帧!)	★ 无法变形、缩放	★ 仅传输数据, 无计算

3.3 OpenGL 着色器语言 GLSL

OpenGL提供的API让我们可以一定程度上控制GPU，比如：Update更新纹理图片等。但是，如果我们想控制GPU执行我们自定义的程序，该如何处理呢？这就需要GLSL语言了。我们可以通过OpenGL提供的API去控制GPU执行我们希望执行的GLSL程序。

GLSL 类似 C 语言，但运行在 GPU 上。我们可以通过它来控制GPU“每个顶点怎么处理，每个像素怎么着色”。

我们要先理解两类着色器：顶点着色器和片元着色器，着色器本质就是代码

- 顶点着色器 (Vertex Shader)用在 **世界坐标系** 中，用于决定“顶点在屏幕上的位置”，可以做平移、旋转、缩放、投影等操作。
- 片元着色器 (Fragment Shader)对应 **纹理坐标系**，用于决定“每个像素显示什么颜色”，可以做纹理采样、光照、颜色混合。

我们来看看示例代码

- 顶点着色器
 - 在顶点着色器中，传入了一个vec4的顶点坐标xyzw，然后直接传递给内建变量gl_Position，即直接根据顶点坐标渲染，不做位置变换。


```
#顶点着色器

attribute vec4 aPosition;

void main() {
    gl_Position = aPosition;
}
```

- 片元着色器

- 在片元着色器中，直接给gl_FragColor赋值，依然是一个vec4类型的数据，这里表示 rgba (Red, Green, Blue, Alpha透明度) 颜色值，这里为完全不透明的红色。

```
#片元着色器

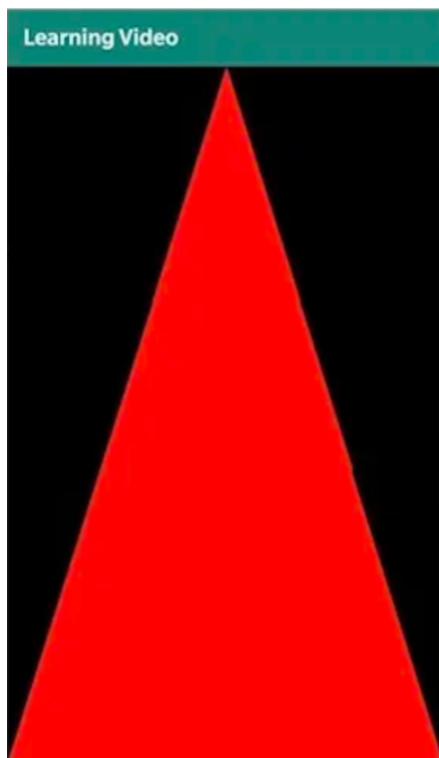
void main() {
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0)
}
```

- 这样，两个简单的着色器串联起来后，每一个片元（像素）都会被填充为红色，最后屏幕会显示一个红色的画面。

3.4 Android OpenGL渲染流程_其一：代码架构设计

我们先来看一下我们的需求：绘制一个红色的三角形

- 具体如下



好，我们怎么设计我们代码的架构呢？

- 架构如下





- 第一层: SimpleRenderActivity + GLSurfaceView构成的界面展示层

- 职责: 提供Activity和View。同时初始化Surface以及EGL相关。并通过GL线程自动执行其绘制的整个生命周期
- 这里的核心是初始化渲染器, 这里的setRenderer是重中之重

```

private fun initRender(drawer: IDrawer) {
    gl_surface.setEGLContextClientVersion(2) // 使用OpenGL ES 2.0
    gl_surface.setRenderer(SimpleRender(drawer)) // 设置渲染器
}
  
```

- 渲染器中使用的drawer底层绘制能力是策略模式

```

drawer = if (intent.getIntExtra("type", 0) == 0) {
    TriangleDrawer() // 绘制三角形
} else {
    BitmapDrawer(...) // 绘制位图
}
  
```

- 第二层: SimpleRender (渲染协调层)

- 职责: 响应GLSurfaceView的生命周期回调
- 设计理念: 他通过自身的回调方法控制渲染的整个流程。Renderer只负责协调, 不负责具体绘制, 其通过委托模式将绘制任务交给Drawer
- 他的回调分为下面的三个: 创建, 尺寸适配, 持续渲染
- onSurfaceCreated - 一次性初始化

```

override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
    GLES20.glClearColor(0f, 0f, 0f, 0f) // 设置清屏颜色 (黑色透明)
    GLES20.glClear(GLES20.GL_COLOR_BUFFER_BIT) // 清空颜色缓冲区
    mDrawer.setTextureID(OpenGLTools.createTextureIds(1)[0]) // 创建纹理ID
}
  
```

- onSurfaceChanged - 尺寸适配

- ```
override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {
 GLES20.glViewport(0, 0, width, height) // 设置视口大小
}
```

- onDrawFrame - 持续渲染

- ```
override fun onDrawFrame(gl: GL10?) {
    mDrawer.draw() // 委托绘制器执行绘制
}
```

- 第三层: IDrawer (绘制抽象层)

- 职责: 这一层定义统一的绘制的能力
- 能力如下:

- ```
interface IDrawer {
 fun draw() // 执行绘制
 fun setTextureID(id: Int) // 设置纹理ID
 fun release() // 释放资源
}
```

- 设计模式: 通过这个接口我们可以实现: 1. 策略模式, 不同绘制策略可互换。 2. 委托模式, Renderer委托Drawer执行绘制。
- 此时, 假如我们想实现不同的绘制策略, 那么更改为不同接口实现即可

- ```

graph TD
    Renderer -- 依赖 --> IDrawer
    IDrawer <--| 实现 | TriangleDrawer
    IDrawer <--| 实现 | BitmapDrawer
    IDrawer <--| 实现 | VideoDrawer
    
```

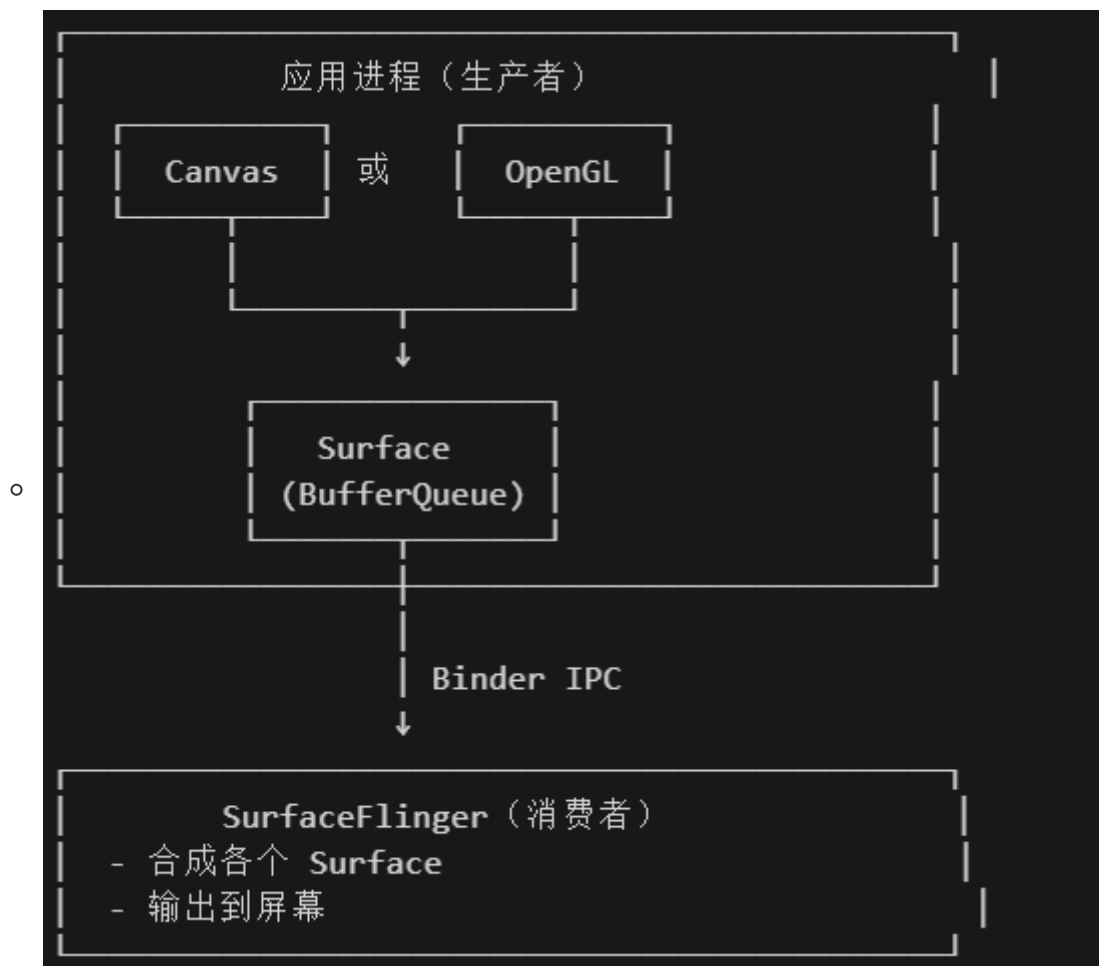
- 第四层: TriangleDrawer (具体实现层)

- 职责: 负责实际的顶点和纹理数据准备, 顶点和纹理着色器程序创建, 实际的绘制执行。
- 具体的代码, 我们后面结合着说。

我们现在有几个问题。疑问一: Surface、SurfaceView、GLSurfaceView的关系? 疑问二: setRenderer到底做了什么? 我们逐个来解答

疑问一: Surface、SurfaceView、GLSurfaceView的关系

- Surface本质是对BufferQueue的封装, BufferQueue是一个GraphicFrame队列。内部的每pop出来的一个都是一个GraphicFrame, 可以存储一帧
 - 本质是跨进程的共享内存
 - 他是生产者-消费者模型, 生产者: 应用进程 (Canvas/OpenGL绘制)。消费者: SurfaceFlinger (合成显示)
 - 通过Binder进程可以通知SurfaceFlinger读取数据显示到屏幕上



- SurfaceView 是持有独立的Surface引用的View。他独立窗口层，不在View树绘制流程中，其使用子线程绘制，不阻塞主线程。
- GLSurfaceView继承SurfaceView，在其基础上封装了GLThread - 独立的渲染线程，EGLHelper - EGL相关，Renderer回调 - 标准化生命周期。（这个详细的我们要在setRenderer中看），如下

```

class GLSurfaceView : SurfaceView {
    // 继承自 SurfaceView，拥有独立 Surface

    // 新增的核心功能：
    private var mGLThread: GLThread // 独立的 OpenGL 渲染线程
    private var mEGLHelper: EGLHelper // EGL 上下文管理器
    private var mRenderer: Renderer // 用户自定义渲染器
}

```

- GLSurfaceView做了什么额外的事情？

- 手动使用EGL的代码

```

// 如果不用 GLSurfaceView，你需要这样写：
val eglDisplay = EGL14.eglGetDisplay(EGL14.EGL_DEFAULT_DISPLAY)
val config = chooseEGLConfig(eglDisplay)
val eglContext = EGL14.eglCreateContext(eglDisplay, config, ...)
val eglSurface = EGL14.eglCreateWindowSurface(eglDisplay, config,
surface, ...)
EGL14.eglMakeCurrent(eglDisplay, eglSurface, eglSurface, eglContext)

// ... OpenGL 绘制 ...

EGL14.eglSwapBuffers(eglDisplay, eglSurface) // 交换缓冲区

```

- GLSurfaceView 封装后

- ```
// 只需要这两行！
glSurfaceView.setEGLContextClientVersion(2)
glSurfaceView.setRenderer(renderer)
```

- 这里的核心是setRenderer。

疑问二：setRenderer到底做了什么？

- 简要看看源码

- ```
// GLSurfaceView.java (简化版)
public void setRenderer(Renderer renderer) {
    checkRenderThreadState(); // 检查不要重复设置

    // 【步骤1】保存 Renderer 引用
    mRenderer = renderer;

    // 【步骤2】创建并启动 GLThread
    mGLThread = new GLThread(mThisweakRef);
    mGLThread.start(); // ← 这里启动了渲染线程！
}
```

- 核心是GLThread 的工作流程，详细如下：

- ```
// GLThread.java (简化版)
class GLThread extends Thread {
 @Override
 public void run() {
 try {
 // =====
 // ☒ 步骤1: 初始化 EGL
 // =====
 mEglHelper = new EglHelper();
 mEglHelper.start(); // 创建 EGL Display

 // =====
 // ☒ 步骤2: 创建 OpenGL 上下文和 Surface
 // =====
 mEglHelper.createGL();
 // 内部执行:
 // - eglChooseConfig()
 // - eglCreateContext()
 // - eglCreateWindowSurface()
 // - eglMakeCurrent()

 // =====
 // ☒ 步骤3: 首次回调 onSurfaceCreated
 // =====
 mRenderer.onSurfaceCreated(gl, mEglHelper.mEglConfig);

 // =====
 // ☒ 步骤4: 回调 onSurfaceChanged
 // =====
 mRenderer.onSurfaceChanged(gl, mwidth, mHeight);

 // =====
 } catch (Exception e) {
 // ...
 }
 }
}
```

```

// ☒ 步骤5: 进入渲染循环
// =====
while (true) {
 if (mPaused) {
 mEglHelper.finish(); // 暂停时释放 EGL
 wait(); // 等待恢复
 continue;
 }

 if (sizeChanged) {
 mRenderer.onSurfaceChanged(gl, w, h);
 sizeChanged = false;
 }

 // 【核心】调用用户的绘制方法
 mRenderer.onDrawFrame(gl);

 // 交换前后缓冲区, 显示到屏幕
 mEglHelper.swap(); // eglSwapBuffers()

 // 控制帧率 (vsync 同步)
 waitForVSync();
}

} finally {
 // 线程结束时清理资源
 mEglHelper.finish();
}
}
}

```

### 3.5 Android OpenGL渲染流程\_其二: TriangleDrawer实现

先看看着色器相关的部分

首先是预准备的代码资源

- 第一: 顶点着色器, 对于一个完整的顶点着色器的处理, 其代码如下

```

○ // 输入: 顶点属性 (从Java层传入)
attribute vec4 aPosition; // 顶点坐标
attribute vec2 aCoordinate; // 纹理坐标

// 输出: 传递给片元着色器的纹理坐标
varying vec2 vTexCoord; // 纹理坐标 (会被GPU自动插值)

void main() {
 // 设置顶点的最终位置 (必须)
 gl_Position = aPosition;

 // 传递纹理坐标给片元着色器
 vTexCoord = aCoordinate;
}

```

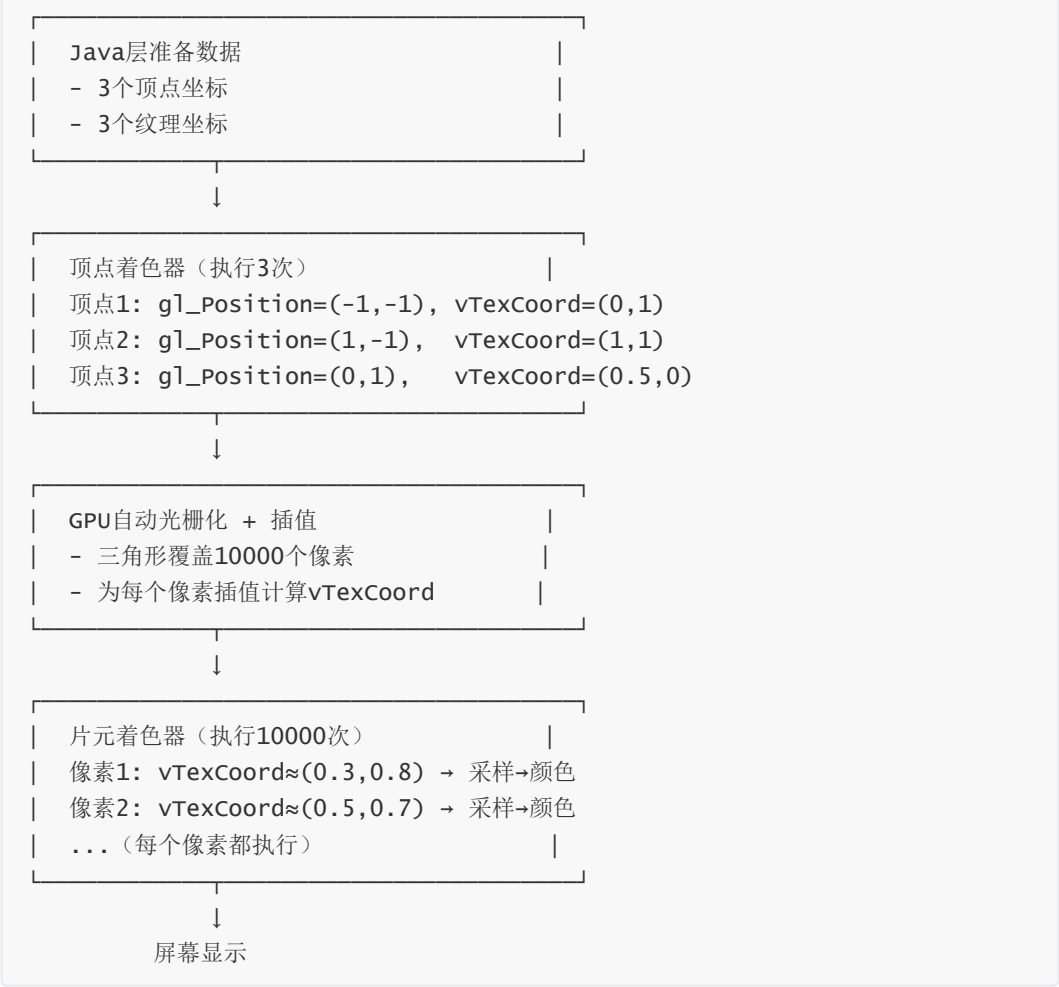
- 第二: 片元着色器, 对于一个完整的片元着色器的处理, 其代码如下

```
// 输入：从顶点着色器传来的数据（已插值）
varying vec2 vTexCoord; // 纹理坐标

// 输入：全局uniform变量
uniform sampler2D uTexture; // 纹理采样器

void main() {
 // 从纹理采样颜色，并设置为最终像素颜色
 gl_FragColor = texture2D(uTexture, vTexCoord);
}
```

- 我们来看看他们配合下的执行流程，具体如下



- 看看着色器变量的类型

- attribute (属性)：顶点着色器的输入，如下

```
attribute vec4 aPosition; // 每个顶点的位置不同
attribute vec2 aCoordinate; // 每个顶点的纹理坐标不同
```

- o uniform (统一变量)：全局常量，对所有顶点/片元都相同，只能读取，不能修改。如下

```
uniform mat4 uMatrix; // 变换矩阵（所有顶点用同一个）
uniform sampler2D uTexture; // 纹理对象（所有像素用同一个）
```

- o varying (变化量): 顶点着色器 → 片元着色器传递数据, 其是GPU自动插值, 如下

- ```
// 顶点着色器中
varying vec2 vTexCoord;
vTexCoord = aCoordinate; // 设置值

// 片元着色器中
varying vec2 vTexCoord; // 接收插值后的值
```

上面是着色器代码资源，看看项目中实际对着色器的操作

- 第一步：loadShader方法，这是着色器的创建和编译方法

- ```
private fun loadShader(type: Int, shaderCode: String): Int {
 //步骤1. 根据type创建顶点着色器或者片元着色器
 /**
 * GL_ES20.GL_VERTEX_SHADER: 创建顶点着色器
 * GL_ES20.GL_FRAGMENT_SHADER: 创建片元着色器
 */
 val shader = GLES20.glCreateShader(type)

 //步骤2. 将资源加入到着色器中，并编译
 GLES20.glShaderSource(shader, shaderCode)
 GLES20.glCompileShader(shader)

 return shader
}
```

- 分为两个过程，创建着色器，将code加载进着色器中。
- 创建着色器用于在GPU显存中分配空间
- ...

GPU端：

|         |               |
|---------|---------------|
| 创建着色器对象 | ← 在GPU显存中分配空间 |
| ID: 1   |               |

Java端：

shader = 1 ← 保存ID引用

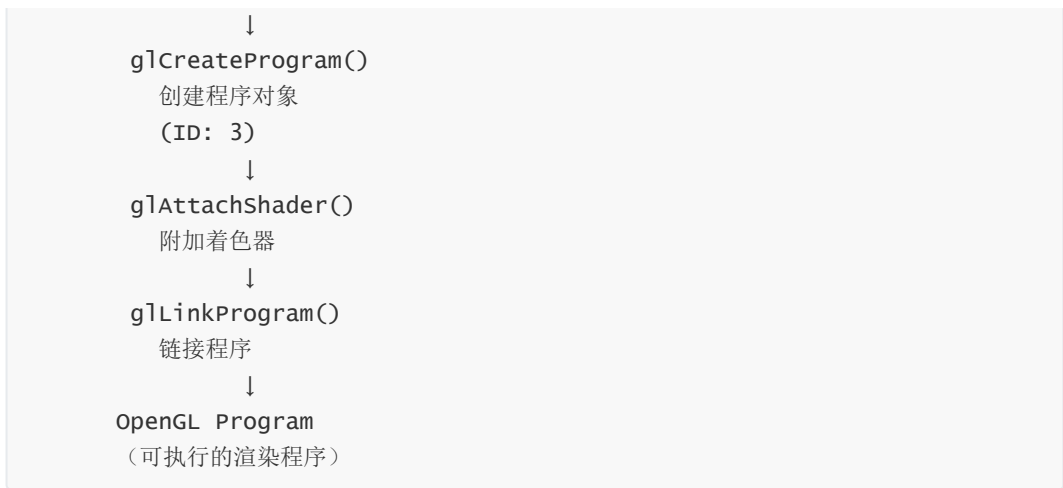
- 加载则是将GLSL源码上传到GPU

- ```
val shaderCode = "attribute vec4 aPosition;" +
    "void main() {" +
    "    gl_Position = aPosition;" +
    "}"
GLES20.glShaderSource(shader, shaderCode)
```

- 编译则是GPU将GLSL源码编译成GPU可执行的机器码，类比gcc编译C代码 → 二进制可执行文件

- 第二步：看看链接的过程

- | | |
|---------------|---------------|
| 顶点着色器 (ID: 1) | 片元着色器 (ID: 2) |
| ↓ | ↓ |
| └──────────┘ | |



- 第三步：获取链接后程序中的变量句柄

- ```
mVertexPosHandler = GLES20.glGetAttribLocation(mProgram, "aPosition")
mTexturePosHandler = GLES20.glGetAttribLocation(mProgram, "aCoordinate")
```

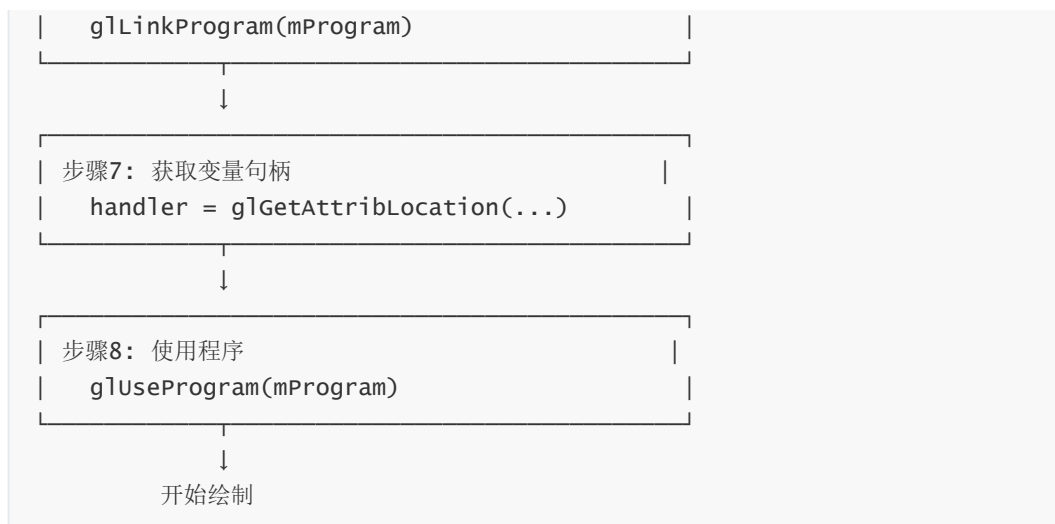
- 其获取着色器中attribute变量的位置索引，用于后续传递顶点数据

- 第四步：使用程序

- `GLES20.glUseProgram(mProgram)`，激活这个OpenGL程序，后续的绘制命令会使用这个程序的着色器

- 小结，这是完整流程





再看看顶点和片元数据的传递

- 先看看代码

```

private fun initPos() {
 // ① 分配直接内存（堆外内存）
 val bb = ByteBuffer.allocateDirect(mVertexCoords.size * 4)

 // ② 设置字节序为本地图
 bb.order(ByteOrder.nativeOrder())

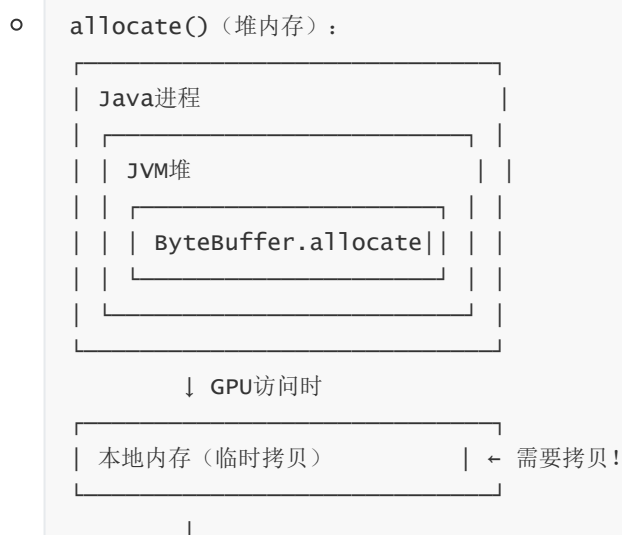
 // ③ 转换为FloatBuffer
 mVertexBuffer = bb.asFloatBuffer()

 // ④ 写入数据
 mVertexBuffer.put(mVertexCoords)

 // ⑤ 重置读取位置
 mVertexBuffer.position(0)
}

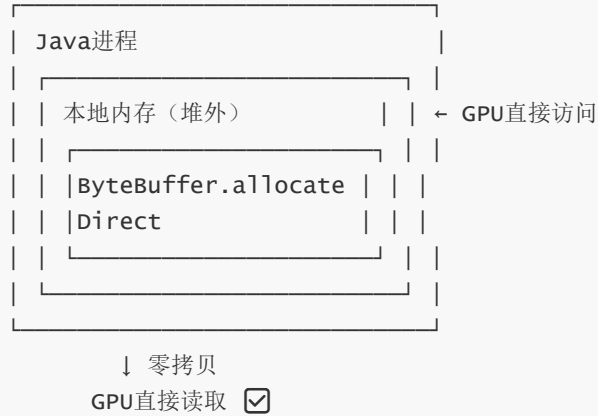
```

- 我们已经有mVertexCoords作为数据，为什么还要再分配直接内存？
- 很简单，因为mVertexCoords是new的，是堆内的内存，GPU没法直接访问需要拷贝访问。因此我们需要直接内存，将数据拷贝进去，让GPU可以直接访问
- 这是内存布局对比



## GPU读取

allocateDirect() (直接内存):



- 看看完整的数据传递流程

```
private fun doDraw() {
 // 1. 启用顶点属性数组
 GLES20.glEnableVertexAttribArray(mVertexPosHandler)
 GLES20.glEnableVertexAttribArray(mTexturePosHandler)

 // 2. 传递顶点数据到GPU
 GLES20.glVertexAttribPointer(
 mVertexPosHandler, // 位置索引
 2, // 每个顶点2个分量(x, y)
 GLES20.GL_FLOAT, // 数据类型: float
 false, // 不需要归一化
 0, // 步长: 0=紧密排列
 mVertexBuffer // 数据源: DirectBuffer
)

 GLES20.glVertexAttribPointer(
 mTexturePosHandler,
 2,
 GLES20.GL_FLOAT,
 false,
 0,
 mTextureBuffer
)

 // 3. 执行绘制
 GLES20.glDrawArrays(GLES20.GL_TRIANGLE_STRIP, 0, 3)
}

glVertexAttribPointer(
 index, // 着色器中attribute变量的位置 (通过glGetAttribLocation获取)
 size, // 每个顶点的分量数 (1, 2, 3, 4)
 type, // 数据类型 (GL_FLOAT, GL_INT等)
 normalized, // 是否归一化 (false: 保持原值)
 stride, // 步长: 下一个顶点的字节偏移 (0=紧密排列)
 buffer // 数据源: 必须是DirectBuffer
)
```

好，我们来看看实际的绘制命令

- 如下

```
private fun doDraw() {
 // 1. 启用顶点属性
 GLES20.glEnableVertexAttribArray(mVertexPosHandler)
 GLES20.glEnableVertexAttribArray(mTexturePosHandler)

 // 2. 传递数据
 GLES20.glVertexAttribPointer(mVertexPosHandler, 2, GLES20.GL_FLOAT,
 false, 0, mVertexBuffer)
 GLES20.glVertexAttribPointer(mTexturePosHandler, 2,
 GLES20.GL_FLOAT, false, 0, mTextureBuffer)

 // 3. 执行绘制
 GLES20.glDrawArrays(GLES20.GL_TRIANGLE_STRIP, 0, 3)
}
```

- `glEnableVertexAttribArray`启用顶点属性。`glVertexAttribPointer`标记对应的属性句柄使用 `mVertexBuffer`这个直接内存中的数据
- `GLES20.glDrawArrays(mode, first, count)` 执行绘制，`mode`是图元装配模式，`first`是起始顶点索引，`count`是顶点数量

感觉真像是你用cpu去控制GPU编译运行程序啊。

最后来看看资源的释放

- **OpenGL资源在GPU显存中，不受Java GC管理**，所以必须手动释放
- 看看`release`方法

```
override fun release() {
 // 1. 禁用顶点属性数组
 GLES20.glDisableVertexAttribArray(mVertexPosHandler)
 GLES20.glDisableVertexAttribArray(mTexturePosHandler)

 // 2. 解绑纹理
 GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, 0)

 // 3. 删除纹理对象
 GLES20.glDeleteTextures(1, intArrayOf(mTextureId), 0)

 // 4. 删除着色器程序
 GLES20.glDeleteProgram(mProgram)
}
```

## 3.6 Android OpenGL渲染流程\_其三：完整代码

完整代码如下：

- 显示层

```
class SimpleRenderActivity : AppCompatActivity() {
 //自定义的OpenGL渲染器，详情请继续往下看
 lateinit var drawer: IDrawer

 override fun onCreate(savedInstanceState: Bundle?) {
```

```

 super.onCreate(savedInstanceState)
 setContentView(R.layout.activity_simpler_render)

 drawer = if (intent.getIntExtra("type", 0) == 0) {
 TriangleDrawer()
 } else {

 BitmapDrawer(BitmapFactory.decodeResource(CONTEXT!!.resources,
 R.drawable.cover))
 }
 initRender(drawer)
 }

 private fun initRender(drawer: IDrawer) {
 gl_surface.setEGLContextClientVersion(2)
 gl_surface.setRenderer(SimpleRender(drawer))
 }

 override fun onDestroy() {
 drawer.release()
 super.onDestroy()
 }
}

<?xml version="1.0" encoding="utf-8"?>
<android.support.constraint.ConstraintLayout
 xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent">
 <android.opengl.GLSurfaceView
 android:id="@+id/gl_surface"
 android:layout_width="match_parent"
 android:layout_height="match_parent"/>
</android.support.constraint.ConstraintLayout>

```

- 渲染协调层

- ```

class SimpleRender(private val mDrawer: IDrawer):
    GLSurfaceView.Renderer {

        override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
            GLES20.glClearColor(0f, 0f, 0f, 0f)
            GLES20.glClear(GLES20.GL_COLOR_BUFFER_BIT)
            mDrawer.setTextureID(OpenGLTools.createTextureIds(1)[0])
        }

        override fun onSurfaceChanged(gl: GL10?, width: Int, height: Int) {
            GLES20.glViewport(0, 0, width, height)
        }

        override fun onDrawFrame(gl: GL10?) {
            mDrawer.draw()
        }
    }

```

- 绘制器接口，抽象绘制器应具备的能力

```
interface IDrawer {  
    fun draw()  
    fun setTextureID(id: Int)  
    fun release()  
}
```

- 绘制器实现层，负责准备数据，创建程序，执行绘制

```
class TriangleDrawer(private val mTextureId: Int = -1): IDrawer {  
    //顶点坐标  
    private val mVertexCoors = floatArrayOf(  
        -1f, -1f,  
        1f, -1f,  
        0f, 1f  
    )  
  
    //纹理坐标  
    private val mTextureCoors = floatArrayOf(  
        0f, 1f,  
        1f, 1f,  
        0.5f, 0f  
    )  
  
    //纹理ID  
    private var mTextureId: Int = -1  
  
    //OpenGL程序ID  
    private var mProgram: Int = -1  
  
    // 顶点坐标接收者  
    private var mVertexPosHandler: Int = -1  
    // 纹理坐标接收者  
    private var mTexturePosHandler: Int = -1  
  
    private lateinit var mVertexBuffer: FloatBuffer  
    private lateinit var mTextureBuffer: FloatBuffer  
  
    init {  
        //【步骤1：初始化顶点坐标】  
        initPos()  
    }  
  
    private fun initPos() {  
        val bb = ByteBuffer.allocateDirect(mVertexCoors.size * 4)  
        bb.order(ByteOrder.nativeOrder())  
        //将坐标数据转换为FloatBuffer，用以传入给OpenGL ES程序  
        mVertexBuffer = bb.asFloatBuffer()  
        mVertexBuffer.put(mVertexCoors)  
        mVertexBuffer.position(0)  
  
        val cc = ByteBuffer.allocateDirect(mTextureCoors.size * 4)  
        cc.order(ByteOrder.nativeOrder())  
        mTextureBuffer = cc.asFloatBuffer()  
        mTextureBuffer.put(mTextureCoors)  
        mTextureBuffer.position(0)  
    }  
}
```

```

    }

    override fun setTextureID(id: Int) {
        mTextureId = id
    }

    override fun draw() {
        if (mTextureId != -1) {
            // 【步骤2：创建、编译并启动OpenGL着色器】
            createGLPrg()
            // 【步骤3：开始渲染绘制】
            doDraw()
        }
    }

    private fun createGLPrg() {
        if (mProgram == -1) {
            val vertexShader = loadShader(GLES20.GL_VERTEX_SHADER,
            getVertexShader())
            val fragmentShader = loadShader(GLES20.GL_FRAGMENT_SHADER,
            getFragmentShader())

            //创建OpenGL ES程序，注意：需要在OpenGL渲染线程中创建，否则无法渲染
            mProgram = GLES20.glCreateProgram()
            //将顶点着色器加入到程序
            GLES20.glAttachShader(mProgram, vertexShader)
            //将片元着色器加入到程序中
            GLES20.glAttachShader(mProgram, fragmentShader)
            //连接到着色器程序
            GLES20.glLinkProgram(mProgram)

            mVertexPosHandler = GLES20.glGetAttribLocation(mProgram,
            "aPosition")
            mTexturePosHandler = GLES20.glGetAttribLocation(mProgram,
            "aCoordinate")
        }
        //使用OpenGL程序
        GLES20.glUseProgram(mProgram)
    }

    private fun doDraw() {
        //启用顶点的句柄
        GLES20.glEnableVertexAttribArray(mVertexPosHandler)
        GLES20.glEnableVertexAttribArray(mTexturePosHandler)
        //设置着色器参数
        GLES20.glVertexAttribPointer(mVertexPosHandler, 2,
        GLES20.GL_FLOAT, false, 0, mVertexBuffer)
        GLES20.glVertexAttribPointer(mTexturePosHandler, 2,
        GLES20.GL_FLOAT, false, 0, mTextureBuffer)
        //开始绘制
        GLES20.glDrawArrays(GLES20.GL_TRIANGLE_STRIP, 0, 4)
    }

    override fun release() {
        GLES20.glDisableVertexAttribArray(mVertexPosHandler)
        GLES20.glDisableVertexAttribArray(mTexturePosHandler)
        GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, 0)
        GLES20.glDeleteTextures(1, intArrayOf(mTextureId), 0)
    }

```

```

        GLES20.glDeleteProgram(mProgram)
    }

    private fun getVertexShader(): String {
        return "attribute vec4 aPosition;" +
            "void main() {" +
            "    gl_Position = aPosition;" +
            "}"
    }

    private fun getFragmentShader(): String {
        return "precision mediump float;" +
            "void main() {" +
            "    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0);" +
            "}"
    }

    private fun loadShader(type: Int, shaderCode: String): Int {
        //根据type创建顶点着色器或者片元着色器
        val shader = GLES20.glCreateShader(type)
        //将资源加入到着色器中，并编译
        GLES20.glShaderSource(shader, shaderCode)
        GLES20.glCompileShader(shader)

        return shader
    }
}

```

小结一下，看一下主要的渲染流程。在 Android 中，OpenGL ES 渲染主要流程如下：

- **初始化**：创建 GLSurfaceView 并设置渲染器（也就是生命周期回调）。
 - 准备着色器相关代码和数据
 - **准备数据**：顶点坐标、纹理坐标、纹理 ID。
 - **准备着色器**：顶点着色器控制顶点位置，片元着色器控制像素颜色。
 - **编译和绑定着色器**：将 GLSL 代码编译、连接成程序并启用。
 - **传入顶点和纹理数据**：通过属性绑定到 GPU。
- 实际渲染
 - **循环渲染**：onDrawFrame 每帧回调执行，GPU 并行计算每个顶点和像素。
 - **绘制**：调用 glDrawArrays 或 glDrawElements 执行渲染。

3.3 Android OpenGL渲染流程_其四：纹理基础知识

我们上面绘制三角形是让所有的像素都显示统一的颜色。如果我们希望这个点显示一种，另一个点显示其他的，该怎么处理呢？

此时就要用到纹理和采样。我们可以把一张位图(Bitmap)的像素内容上传到 GPU，作为“采样源”，然后在片元着色器中使用接收到的被插值的纹理坐标从纹理中对应坐标获取颜色放置到对应的屏幕中的像素的位置上。这样我们就可以将图片上的色彩显示到屏幕上了。

好，这里我们要补充一下纹理的基础知识：纹理对象、纹理单元、纹理目标

- **纹理对象**：本质是GPU显存中存储的一块图像数据


```

纹理对象 = {
    ID: 整数 (如5、7、9)      ← OpenGL的身份证号
    宽度: 1024                ← 图像宽度 (像素)
    高度: 768                 ← 图像高度 (像素)
    格式: RGBA                ← 像素格式
    数据: [像素数组]          ← 实际的图像数据
    参数: {                    ← 纹理参数
        过滤方式: LINEAR
        环绕方式: CLAMP_TO_EDGE
        ...
    }
}

```

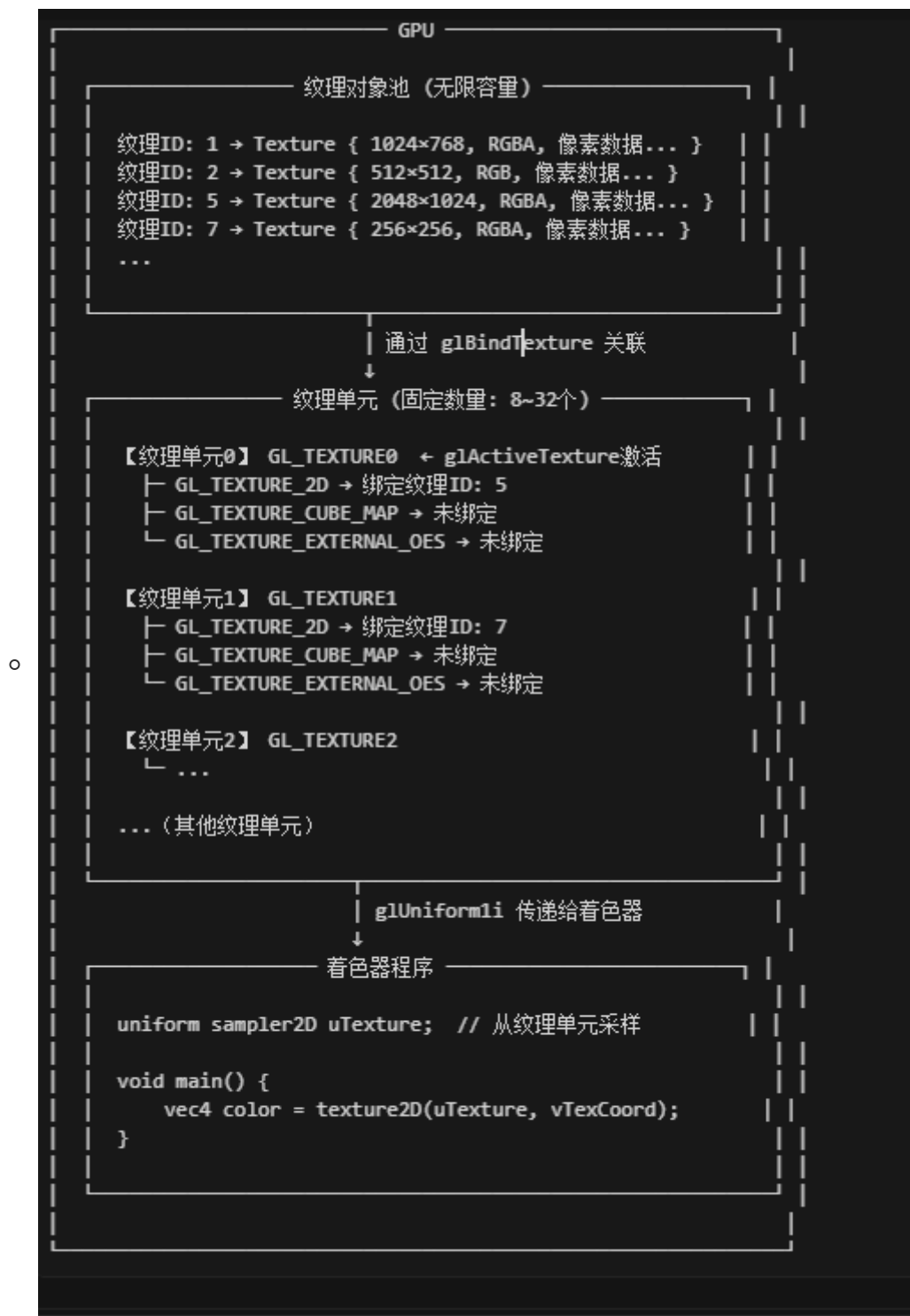
- 纹理单元: GPU的工作插槽, 你的上面的纹理对象如果想被处理, 那么要先被放到这个纹理单元里
 - 为什么需要多个单元? 让着色器可以从多个纹理单元同时采样, 避免频繁切换纹理, 提高性能
 - 感觉类似多线程处理多个数据
- 纹理目标: 纹理目标就像是插槽的类型, 不同类型的插槽接受不同类型的设备
 - 常见有下面的类型

常见的纹理目标:

- - GL_TEXTURE_2D → 二维纹理 (普通图片)
 - GL_TEXTURE_CUBE_MAP → 立方体贴图 (天空盒)
 - GL_TEXTURE_EXTERNAL_OES → 外部纹理 (相机输入、视频)
- 为什么要这样? 因为不同类型的纹理数据结构不同, 同时GPU处理方式不同

好, 我们先看看他的整体的架构

- 我们通过glGenTextures从纹理对象池中申请的GPU纹理对象, 通过glBindTexture和纹理单元关联。我们通过glActiveTexture激活纹理单元。



好，我们来看看他的完整的工作流程：从创建到使用

- 第一步：创建纹理对象
 - OpenGL从可用ID池中分配一个整数（如5），此时就是有了纹理对象，但是没有实际的存储纹理数据的内存空间。

```
// 1. 生成纹理ID
int[] textureIds = new int[1];
◦ GL ES20.glGenTextures(1, textureIds, 0);
int textureId = textureIds[0]; // 假设得到ID: 5

// 🚀 此时GPU说：我给你分配了ID=5，但这个纹理还是空的
```

- 第二步：激活纹理单元
 - 选择当前要操作的纹理单元，后续的 `glBindTexture` 会绑定到这个单元

```
// 2. 激活纹理单元0
GLS20.glActiveTexture(GLS20.GL_TEXTURE0);

// 🚩 此时GPU说：当前工作在纹理单元0，后续操作都针对这个单元
```

- 第三步：绑定纹理到目标

- 纹理单元0的GL_TEXTURE_2D目标，现在绑定了纹理5，类似于现在选中对象
- 其实就是建立了索引关系
- 绑定后，后续操作（配置参数、上传数据）都作用于这个纹理

```
// 3. 绑定纹理ID到2D目标
GLS20.glBindTexture(GLS20.GL_TEXTURE_2D, textureId);

// 🚩 此时GPU说：纹理单元0的GL_TEXTURE_2D目标，现在绑定了纹理5
```

- 第四步：配置纹理参数（设置照片的显示方式）

```
// 4. 配置纹理参数
GLS20.glTexParameterf(GLS20.GL_TEXTURE_2D,
    GLS20.GL_TEXTURE_MIN_FILTER,
    GLS20.GL_LINEAR); // 缩小过滤：线性插值

GLS20.glTexParameterf(GLS20.GL_TEXTURE_2D,
    GLS20.GL_TEXTURE_MAG_FILTER,
    GLS20.GL_LINEAR); // 放大过滤：线性插值

GLS20.glTexParameterf(GLS20.GL_TEXTURE_2D,
    GLS20.GL_TEXTURE_WRAP_S,
    GLS20.GL_CLAMP_TO_EDGE); // S轴(横向)环绕：边缘截断

GLS20.glTexParameterf(GLS20.GL_TEXTURE_2D,
    GLS20.GL_TEXTURE_WRAP_T,
    GLS20.GL_CLAMP_TO_EDGE); // T轴(纵向)环绕：边缘截断
```

- 第五步：上传纹理数据（把照片内容传给GPU）

```
// 5. 上传位图数据到GPU
Bitmap bitmap = BitmapFactory.decodeResource(getResources(), R.drawable.photo);
GLUtils.texImage2D(GLS20.GL_TEXTURE_2D, 0, bitmap, 0);

// 或者手动方式：
// GLS20.glTexImage2D(GLS20.GL_TEXTURE_2D, 0, GLS20.GL_RGBA,
//     width, height, 0, GLS20.GL_RGBA,
//     GLS20.GL_UNSIGNED_BYTE, buffer);

bitmap.recycle(); // 释放CPU内存中的位图

// 🚩 此时GPU说：纹理5现在有完整的数据了，可以使用了
```

- GPU内部操作：

- 分配显存空间：根据宽高和格式计算需要的显存大小
- 上传像素数据：从CPU内存复制到GPU显存
- 将指针记录到纹理对象的数据索引中

- GPU中存储如下



- 这一步有纹理对象有内存了。

3.3 Android OpenGL渲染流程_其五：使用纹理显示图片

接下来，我们用代码来看看我们如何使用纹理。我们先看看之前被我们忽略的代码：

初始化纹理资源

- 我们在Surface创建阶段，使用createTextureIds创建了纹理ID，并使用setTextureID设置

```
class SimpleRender(private val mDrawer: IDrawer): GLSurfaceView.Renderer {
    override fun onSurfaceCreated(gl: GL10?, config: EGLConfig?) {
        GLES20.glClearColor(0f, 0f, 0f, 0f)
        GLES20.glClear(GLES20.GL_COLOR_BUFFER_BIT)
        mDrawer.setTextureID(OpenGLTools.createTextureIds(1)[0])
    }
}
```

- 看看createTextureIds，申请纹理ID并返回纹理ID

```
fun createTextureIds(count: Int): IntArray {
    val texture = IntArray(count)
    GLES20.glGenTextures(count, texture, 0) //生成纹理
    return texture
}
```

- 看看setTextureID，把生成的纹理ID保存到成员变量，后续绘制时会用到

```
override fun setTextureID(id: Int) {
    mTextureId = id
}
```

接着是纹理状态下数据的准备

- 和三角形不同，这里需要一个矩形来显示整张图片：
- 顶点坐标

```
// 顶点坐标
private val mVertexCoords = floatArrayOf(
    -1f, -1f,
    1f, -1f,
    -1f, 1f,
    1f, 1f
)
```

- 纹理坐标

```

// 纹理坐标
private val mTextureCoors = floatArrayOf(
    0f, 1f,
    1f, 1f,
    0f, 0f,
    1f, 0f
)

```

- 这里最终仍然需要通过 `FloatBuffer` 封装传给 GPU。
- 会不会好奇为什么坐标是这样的，就不能是 `0f,1f 0f,0f 1f,1f 1f,0f` 坐标吗？我们回头再说

准备着色器

- 我们的顶点着色器和片元着色器和上面不一样

```

private fun getVertexShader(): String {
    return "attribute vec4 aPosition;" +
        "attribute vec2 aCoordinate;" +
        "varying vec2 vCoordinate;" +
        "void main() {" +
        "    gl_Position = aPosition;" +
        "    vCoordinate = aCoordinate;" +
        "}"
}

private fun getFragmentShader(): String {
    return "precision mediump float;" +
        "uniform sampler2D uTexture;" +
        "varying vec2 vCoordinate;" +
        "void main() {" +
        "    vec4 color = texture2D(uTexture, vCoordinate);" +
        "    gl_FragColor = color;" +
        "}"
}

```

编译和绑定着色器

- 这个过程和我们之前一样，因为其实我们只改变了顶点着色器和片元着色器的代码，最终同样是加入到 GLSL 程序中。
- 不过，我们这里要获取的属性的索引更多了，因为我们用到的更多了

```

private fun createGLPrg() {
    if (mProgram == -1) {
        //省略与绘制三角形一致的部分
        //.....

        mVertexPosHandler = GLES20.glGetAttribLocation(mProgram, "aPosition")
        mTexturePosHandler = GLES20.glGetAttribLocation(mProgram, "aCoordinate")
        //【注3：新增获取纹理接收者】
        mTextureHandler = GLES20.glGetUniformLocation(mProgram, "uTexture")
    }
    //使用OpenGL程序
    GLES20.glUseProgram(mProgram)
}

```

传入顶点和纹理数据

- 相对于之前的，我们要先进行两步初始化操作

```

fun draw() {
    if (mTextureId != -1) {
        //【步骤2：创建、编译并启动OpenGL着色器】
        createGLPrg()
        //-----【注4：新增两个步骤】-----
        //【步骤3：激活并绑定纹理单元】
        activateTexture()
        //【步骤4：绑定图片到纹理单元】
        bindBitmapToTexture()
        //-----
    }
}

```

- 先看看activateTexture
 - 激活0号纹理单元
 - 把mTextureId这个2D纹理对象绑定到当前激活的纹理单元。
 - 把当前使用的纹理单元编号（这里是0，对应GL_TEXTURE0）传递给着色器中的uTexture，也就是让着色器知道要采样哪个纹理单元
 - 配置过滤与环绕

```

private fun activateTexture() {
    //激活指定纹理单元
    GLES20.glActiveTexture(GLES20.GL_TEXTURE0)
    //绑定纹理ID到纹理单元
    GLES20.glBindTexture(GLES20.GL_TEXTURE_2D, mTextureId)
    //将激活的纹理单元传递到着色器里面
    GLES20.glUniform1i(mTextureHandler, 0)
    //配置边缘过滤参数
    GLES20.glTexParameterf(GLES20.GL_TEXTURE_2D, GLES20.GL_TEXTURE_MIN_FILTER, GLES20.GL_LINEAR)
    GLES20.glTexParameterf(GLES20.GL_TEXTURE_2D, GLES20.GL_TEXTURE_MAG_FILTER, GLES20.GL_LINEAR)
    GLES20.glTexParameteri(GLES20.GL_TEXTURE_2D, GLES20.GL_TEXTURE_WRAP_S, GLES20.GL_CLAMP_TO_EDGE)
    GLES20.glTexParameteri(GLES20.GL_TEXTURE_2D, GLES20.GL_TEXTURE_WRAP_T, GLES20.GL_CLAMP_TO_EDGE)
}

```

- 然后是bindBitmapToTexture
 - 上传 Bitmap 数据到 GPU，也就是我们之前的bitmap

```

private fun bindBitmapToTexture() {
    if (!mBitmap.isRecycled) {
        //绑定图片到被激活的纹理单元
        GLUtils.texImage2D(GLES20.GL_TEXTURE_2D, 0, mBitmap, 0)
    }
}

```

- 然后传入顶点与纹理坐标数据，和上面一致

绘制

- 绘制4个顶点

```

private fun doDraw() {
    //省略与绘制三角形一致的部分
    //.....
    //【注5：绘制顶点加1，变为4】
    //开始绘制：最后一个参数，将顶点数里改为4
    GLES20.glDrawArrays(GLES20.GL_TRIANGLE_STRIP, 0, 4)
}

```

3.5 绘制模式

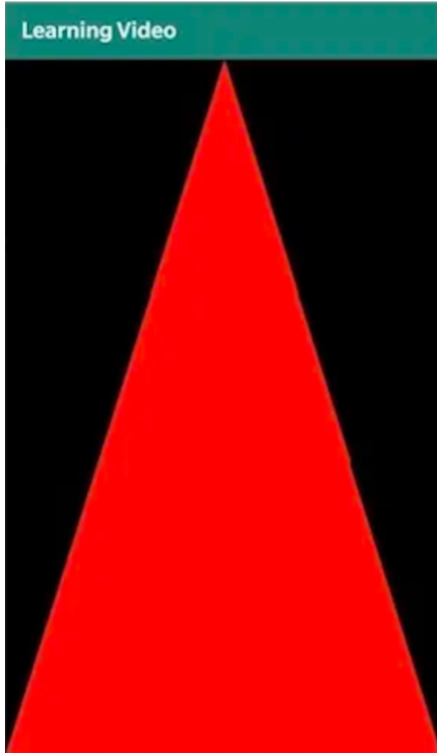
来吧，我们来理解一下这里的GLS20.GL_TRIANGLE_STRIP：绘制方式和3个顶点，以及和下面4个顶点的差异

- 首先是3个顶点绘制

- 这是代码

```
private fun doDraw() {  
    //启用顶点的句柄  
    GLES20.glEnableVertexAttribArray(mVertexPosHandler)  
    GLES20.glEnableVertexAttribArray(mTexturePosHandler)  
    //设置着色器参数， 第二个参数表示一个顶点包含的数据数里，这里为xy，所以为2  
    GLES20.glVertexAttribPointer(mVertexPosHandler, 2, GLES20.GL_FLOAT, false, 0, mVertexBuffer)  
    GLES20.glVertexAttribPointer(mTexturePosHandler, 2, GLES20.GL_FLOAT, false, 0, mTextureBuffer)  
    //开始绘制  
    GLES20.glDrawArrays(GLES20.GL_TRIANGLE_STRIP, 0, 3)  
}
```

- 这是结果



- 然后是4个顶点绘制

- 这是代码

```
private fun doDraw() {  
    //省略与绘制三角形一致的部分  
    //.....  
    //【注5：绘制顶点加1，变为4】  
    //开始绘制：最后一个参数，将顶点数里改为4  
    GLES20.glDrawArrays(GLES20.GL_TRIANGLE_STRIP, 0, 4)  
}
```

- 这是结果



还有啊，4个顶点绘制的时候他的数据是这样的

- 顶点坐标

```
// 顶点坐标
private val mVertexCoords = floatArrayOf(
    -1f, -1f,
    1f, -1f,
    -1f, 1f,
    1f, 1f
)
```

- 纹理坐标

```
// 纹理坐标
private val mTextureCoords = floatArrayOf(
    0f, 1f,
    1f, 1f,
    0f, 0f,
    1f, 0f
)
```

- 会不会好奇为什么坐标是这样的，就不能是0,1 0,0 1,1 1,0吗？我们回头再说

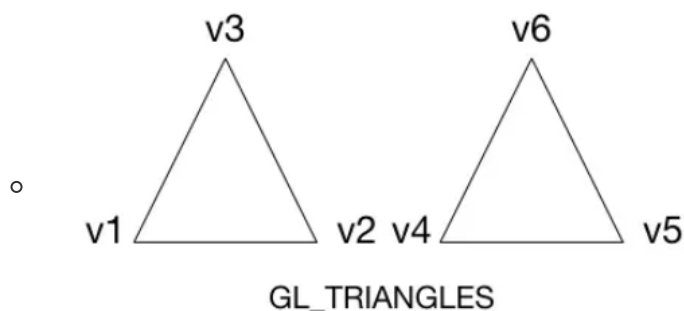
好啊，为什么会这样？

我们先来看第一个问题，绘制模式和3个，4个顶点的问题

OpenGL ES所有的画面都是由三角形构成的，比如一个四边形由两个三角形构成，其他更复杂的图形也都可以分割为大大小小的三角形。

因此，顶点坐标也是根据三角形的连接来设置的。其绘制方式有三种：

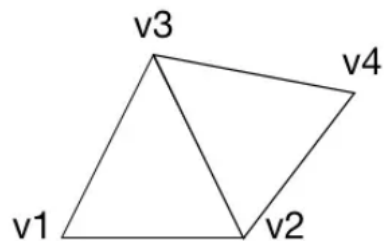
- GL_TRIANGLES：独立顶点的构成三角形



GL_TRIANGLES

- GL_TRIANGLE_STRIP：复用顶点构成三角形

○

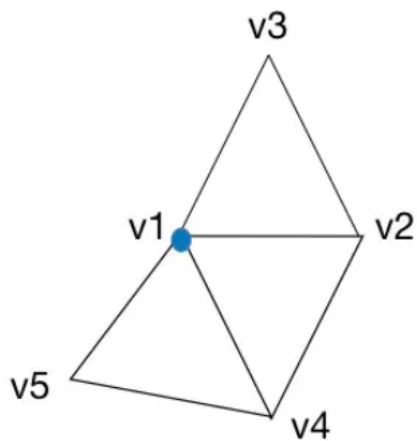


GL_TRIANGLE_STRIP

GL_TRIANGLE_STRIP

- GL_TRIANGLE_FAN: 复用第一个顶点构成三角形

○



GL_TRIANGLE_FAN

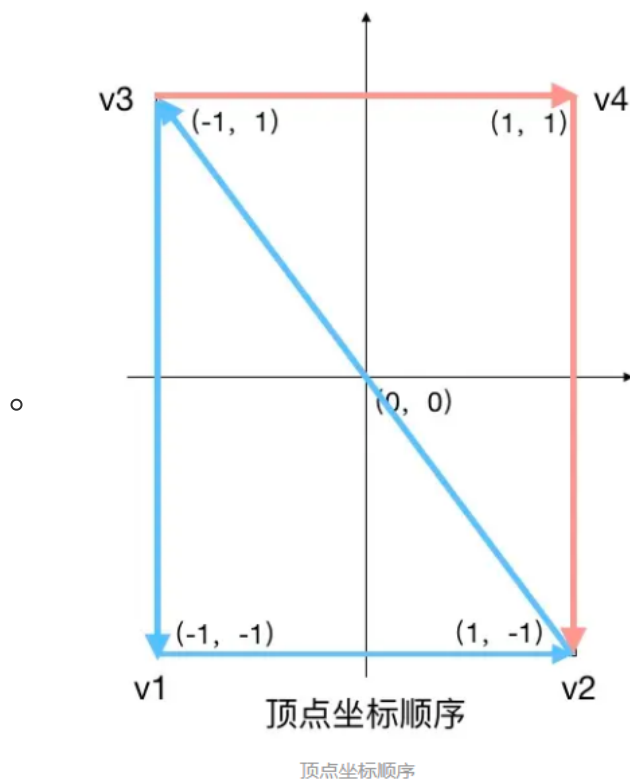
GL_TRIANGLE_FAN

通常情况下，一般使用GL_TRIANGLE_STRIP绘制模式。那么一个四边形的顶点顺序看起来是这样子的
(v1-v2-v3) (v2-v3-v4)

- 也就是下面的

○

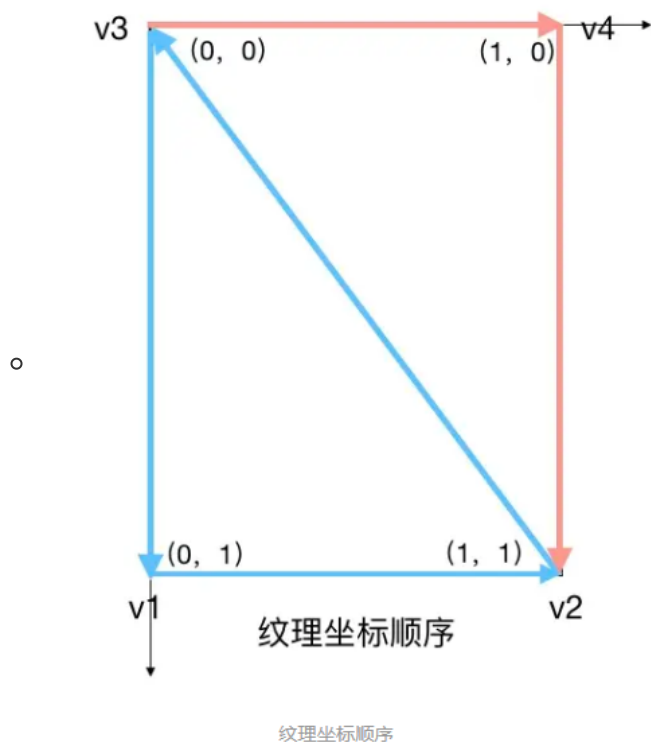
```
// 顶点坐标
private val mVertexCoors = floatArrayOf(
    -1f, -1f,
    1f, -1f,
    -1f, 1f,
    1f, 1f
)
```



对应的纹理坐标也要和顶点坐标顺序一致，否则会出现颠倒，变形等异常

- 也就是下面的

```
// 纹理坐标
private val mTextureCoords = floatArrayOf(
    0f, 1f,
    1f, 1f,
    0f, 0f,
    1f, 0f
)
```



04.底层原理

05.深度思考

5.1 关键问题探究

5.2 设计对比

06.实践验证

6.1 行为验证代码

6.2 性能测试

07.应用场景

7.1 最佳实践

7.2 使用禁忌

08.总结提炼

8.1 核心收获

8.2 知识图谱

8.3 延伸思考

•

09.参考资料

- 1.
- 2.

3.

其他介绍

01.关于我的博客

- csdn: http://my.csdn.net/qq_35829566
- 掘金: <https://juejin.im/user/499639464759898>
- github: <https://github.com/jjjjjjava>
- 邮箱: [934137388@qq.com]